

# **EDT — Engineering Reference Manual**

**Sector7 USA LLC · EDIT/EDT for Linux and Windows**

Engineering reference, July 2026 · V1.05-0

# Table of contents

## EDT — Engineering Reference Manual

- 1 Overview
- 2 Invocation and command line
  - 2.1 DCL form (on a VMS host)
  - 2.2 Host shell form
  - 2.3 Exit codes
  - 2.4 Stdin / stdout / TTY
- 3 The two modes
  - 3.1 Line mode
  - 3.2 Screen / keypad mode
  - 3.3 Switching back to line mode
- 4 Buffer model
  - 4.1 Canonical buffers
  - 4.2 Modified flag
  - 4.3 Line-number model
- 5 Line-mode commands — alphabetical reference
  - ADVANCE
  - APPEND
  - BACKUP
  - BOTTOM
  - C / CHANGE
  - CHGL — Change to lowercase
  - CHGU — Change to uppercase
  - CLEAR
  - COPY
  - CUT
  - DEFINE KEY
  - DEFINE MACRO
  - DELETE
  - DEL C / DC / DELETE CHARACTER
  - DEL EOL
  - DEL L / DL / DELETE LINE
  - DEL W / DW / DELETE WORD
  - EXIT
  - FILL
  - FIND
  - FNDNXT / FIND NEXT
  - GOTO-LINE (synthetic)
  - HELP
  - INCLUDE

INSERT  
MOVE  
NEXT  
PASTE  
PREVIOUS  
PRINT  
QUIT  
REDO  
REPLACE  
RESET  
RESEQUENCE  
SELECT  
SET ...  
SHOW ...  
SUBSTITUTE  
TAB  
TAB ADJUST  
TOP  
TYPE / PRINT (display)  
UNDO  
UND L / UNDELETE LINE  
UND C / UNDELETE CHARACTER  
UND W / UNDELETE WORD  
WRITE  
RANGE  
RETAIN / STORE TEXT

Command qualifiers (slash flags)

## 6 SET and SHOW parameter reference

### 6.1 Still incomplete vs OpenVMS

## 7 Keypad reference (screen mode)

### 7.1 Keypad layouts (VT100 / VT220 and PC)

### 7.2 PF-key bindings

### 7.3 Numeric keypad bindings

### 7.4 Control-key bindings

### 7.5 Cursor / motion keys

### 7.6 GOLD + letter / GOLD + arrow

### 7.7 LK201 editing keypad (E1–E6)

### 7.8 Reference layer (not wired into Editor)

## 8 Search and substitute

### 8.1 Search target syntax

### 8.2 Wildcard set

### 8.3 Direction

- 8.4 Case sensitivity
- 8.5 Wrap-around
- 8.6 Search flags (BEGIN / END / WORD)
- 8.7 Entity / word delimiters
- 8.8 SUBSTITUTE
- 8.9 Per-line repeat counts
- 9 EDTINI.EDT — startup file
  - 9.1 Search order
  - 9.2 Format
  - 9.3 /INIT and /NOINIT
  - 9.4 Example EDT.INI
- 10 Key definitions and macros
  - 10.1 DEFINE KEY reference
  - 10.2 Built-in key ids (canonical strings)
  - 10.3 Examples
  - 10.4 Function-name table
  - 10.5 SHOW KEY
  - 10.6 DEFINE MACRO reference
- 11 Journal / recovery
- 12 Internals
  - 12.1 Top-level structure of edt.cpp
  - 12.2 Line storage
  - 12.3 Terminal I/O
  - 12.4 TTY raw-mode setup
  - 12.5 Signal handling
  - 12.6 Memory limits
- 13 Known divergences from VMS EDT
  - 13.1 Commands missing or stubbed
  - 13.2 Qualifiers missing or partial
  - 13.3 SET/SHOW parameters
  - 13.4 Keypad differences
  - 13.5 Search and substitute
  - 13.6 File system / RMS (deferred)
  - 13.7 Journal / recovery
  - 13.8 DCL / command procedures
  - 13.9 Documented in edt.cpp itself
- 14 Files this build cares about
- 15 End notes
  - 15.1 Build
  - 15.2 Regression suite
  - 15.3 Source statistics
  - 15.4 Acknowledgements

Appendix A — Status line and message reference

Appendix B — Line-mode prompt error reference

Appendix C — Quick reference card

Line mode

Screen mode

Qualifiers

Startup / CLI

# EDT — Engineering Reference Manual

A complete reference for Sector7's EDT-compatible editor (`edt.cpp`, C++17; synced July 2026, **V1.02-0** / 2026-07-15). Covers line-mode commands, the VT100-style keypad in screen mode, the `SET/SHOW` parameter family, the `DEFINE KEY` and `DEFINE MACRO` extensions, the `EDT.INI` startup file, multi-buffer and multi-file editing, the EDT wildcard search engine, recovery and journal status, and every internal class that makes the editor work. Where the implementation diverges from the OpenVMS reference, this manual flags the gap and points at the source line that proves it. **RMS record formats and OpenVMS ;n versioning remain deferred** (see §13.6).

# 1 Overview

EDT — short for the **Editor** — is the historic line-and-screen editor shipped with every release of DEC's VAX/VMS and OpenVMS operating systems from the early 1980s onward. On a real VMS host the editor is invoked from DCL with `$ EDIT/EDT file-spec`; on terminals such as the VT100, VT220 or LK201 it offers a numeric-keypad-driven screen mode in addition to a stdio `*`-prompt command language inherited directly from its predecessor `EDI`.

Two modes are visible to the user:

| Mode            | Entered by                                          | Left by                                     | Input style                  |
|-----------------|-----------------------------------------------------|---------------------------------------------|------------------------------|
| Line            | Editor start (default)                              | <code>EXIT</code> , <code>QUIT</code> , EOF | stdio <code>*</code> prompt  |
| Screen / Keypad | <code>CHANGE</code> or <code>SET MODE CHANGE</code> | <code>Ctrl/Z</code>                         | ncurses/PDCurses full-screen |

The Sector7 build (`edt.cpp`) is a single-translation-unit C++17 program that re-implements EDT for Linux, macOS, and Windows 11. It talks to the terminal through ncurses on POSIX and PDCurses on Windows. The runtime is otherwise self-contained: there is no separate preprocessor, terminal database, or VMS RMS dependency. The line-mode interpreter is the same code path used by `-command` files and by the in-editor `GOLD+7` command prompt.

Sector7's primary customers for this editor are OpenVMS-to-Linux migration teams who want their developers to retain the EDT command vocabulary while they move code off VMS. As a consequence the editor behaves like the OpenVMS reference where practical and diverges only where the host OS forces it (primarily **RMS record formats** and `;n` file versions — intentionally deferred; see §13). The original DEC reference manual (`OVMS_EDT_REF.pdf`) remains authoritative for VMS-spec behaviour; this document is the engineering reference for the **Sector7 re-implementation**.

The implementation is licensed:

```
Copyright Sector7 USA LLC 2026
```

(see the comment header at `edt.cpp:1-4`).

## 2 Invocation and command line

### 2.1 DCL form (on a VMS host)

```
$ EDIT/EDT [/qualifiers] file-spec
```

On a real VMS system EDT exposes a long list of `/QUALIFIER` flags. The Sector7 reimplementation runs under a POSIX shell or under VX/RT's DCL emulator, so the DCL qualifiers are translated by the shell wrapper before they reach `edt.cpp`. The argv parser in `main()` itself (`edt.cpp:4145-4190`) only consumes a tiny subset:

| Qualifier                  | Mapped to                                                         | Implemented?                                  |
|----------------------------|-------------------------------------------------------------------|-----------------------------------------------|
| <code>/COMMAND=file</code> | <code>-command=file</code>                                        | Yes                                           |
| <code>/NOCOMMAND</code>    | (absence of <code>-command</code> )                               | Implicit                                      |
| <code>/INIT[=file]</code>  | <code>EDT_INI</code> env var                                      | Via env only                                  |
| <code>/NOINIT</code>       | unset <code>EDT_INI</code> and remove <code>\$HOME/EDT.INI</code> | Manual                                        |
| <code>/RECOVER</code>      | <code>-recover</code> / <code>/RECOVER</code>                     | Yes — text <code>.jou</code> replay (see §11) |
| <code>/JOURNAL=file</code> | (auto <code>file.jou</code> )                                     | Text append; deleted on clean EXIT            |
| <code>/OUTPUT=file</code>  | <code>-output=</code> / <code>/OUTPUT=</code>                     | Yes                                           |
| <code>/READ_ONLY</code>    | <code>-readonly</code> / <code>/READ_ONLY</code>                  | Yes                                           |
| <code>/CREATE</code>       | (file auto-created on EXIT)                                       | Implicit                                      |

### 2.2 Host shell form

```
edt file [file2 ...] [-command commands.edt | -command=commands.edt]
          [-output=path] [-readonly] [-recover]
```

The complete argv contract is implemented by `main()` (`edt.cpp:4145-4190`) plus `parse_command_arg()` (`edt.cpp:3935-3960`):

- The first argument **must** be a filename — `argc < 2` triggers the usage banner on `stderr` and an `EXIT_FAILURE` `exit` (`edt.cpp:4146-4148`).
- Every subsequent non-option token is added to the multi-file list until the first `-`-prefixed argument is seen (`edt.cpp:4169-4174`). All files become navigable via the `NEXT` and `PREVIOUS` line-mode commands (§5).
- `-command file` or `-command=file` selects a command file replayed immediately after the EDT.INI startup file (`edt.cpp:3935-3960`).
- `-d` or `-k` (also `/DEBUG`) enables **debug key display**: in screen mode the status line shows the decoded name and raw code of the last key pressed (e.g. `KEY: PF1/F1 (code 265 / 0x109)`),

which helps map foreign keyboards and figure out Num Lock. Turn it off by restarting without the switch.

- The `TERM` environment variable must not be `dumb` (`edt.cpp:4154-4159`); the editor refuses to start otherwise because ncurses cannot drive a `dumb` terminal.
- The `VX_TTY` environment variable, if set, names a tty device. All three stdio file descriptors are redirected to it before ncurses initialises (`edt.cpp:3967-4005`). This is the standard hook for running EDT under an IDE that intercepts stdio.

## 2.3 Exit codes

| Code                      | Meaning                                                                                                                 | Source                                |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| 0                         | EXIT: file saved successfully; or QUIT (VMS-compatible <code>\$STATUS = SS\$_NORMAL</code> ); or stdin EOF in line mode | <code>edt.cpp:3379, 3398, 2088</code> |
| 2                         | EXIT path: save failed                                                                                                  | <code>edt.cpp:3376</code>             |
| <code>EXIT_FAILURE</code> | argv error, <code>TERM=dumb</code> , empty file list, bad <code>\$VX_TTY</code> , or fatal exception                    | <code>edt.cpp:4145-4190, 3967</code>  |

## 2.4 Stdin / stdout / TTY

Line mode reads commands from `std::cin` and writes everything to `std::cout/std::cerr`. A real TTY is **not** required for line mode — the editor is fully scriptable via `-command` files and stdin redirection. Screen mode does require a real TTY (or `VX_TTY`) because ncurses cannot initialise against a pipe.

## 3 The two modes

### 3.1 Line mode

Line mode is the default. After loading the file, the editor prints a banner like

```
$ EDIT/EDT myfile.txt
1 The quick brown fox jumps over the lazy dog
*
```

and enters the REPL implemented by `line_mode()` (`edt.cpp:2088-2102`). The leading `*` is the prompt; it can be changed with `SET PROMPT "..."` (see §6). Each line of stdin is read by `std::getline`, parsed by `parse_command()` (`edt.cpp:1435-1590`), dispatched by `dispatch_command()` (`edt.cpp:1739-1945`), and any command result that means "exit" or "switch to screen mode" is acted on by `handle_command_result()` (`edt.cpp:2064-2079`).

Commands are case-insensitive. Verbs, qualifiers and keywords are folded to upper case for comparison; arguments retain their original case unless explicitly upper-cased (e.g. line-range tokens are upper-cased so that `BEGIN` and `begin` are equivalent — `edt.cpp:1361-1385`).

A bare integer at the prompt is recognised as a line-jump: typing `42` moves the cursor to line 42 and types it, regardless of any other verb (`edt.cpp:1446-1456`).

A `!`, `#`, or `;` at the start of a line marks a comment and is silently skipped (`edt.cpp:1302-1314`). Blank lines are skipped.

### 3.2 Screen / keypad mode

The `CHANGE` command (or its alias `C`, or `SET MODE CHANGE`) drops into ncurses screen mode via

`Editor::screen_mode()` (`edt.cpp:2988-3040`):

1. `\033=` puts the terminal into application keypad mode so the numeric keypad sends `ESC O p...` `ESC O y` instead of bare digits.
2. `initscr(); raw(); noecho(); keypad(stdscr, TRUE); nonl();` configures ncurses for raw input. `set_escdelay(25)` shortens the ESC delay so PF-key sequences resolve fast (POSIX only — `edt.cpp:2996-2998`).
3. `define_key()` registers thirteen private key codes (`EDT_KP0 ... EDT_KPE`, defined at `edt.cpp:233-248`) for the VT100 application-keypad escape sequences. Top-row digit keys keep their normal `'0' ... '9'` codes and continue to insert text.
4. The frame loop is

```
while (!done) { draw_screen(); int ch = getch(); done = handle_screen_key(ch);
clamp_cursor(); adjust_view(); }
```

1. On exit (`Ctrl/Z` or a chord that calls `quit_without_saving_now` etc.) `endwin()` is called and `\033>` returns the terminal to normal keypad mode.

The screen-mode UI (`draw_screen()`, `edt.cpp:3283-3361`) shows:

```
<file lines>
---
EDIT[MAIN] ADVANCE OVR SELECT * col+12 PF1=GOLD PF3=FNDNXT GOLD+PF3=FIND Ctrl/
Z=CMD
<message line>
```

- The status line shows the active buffer in square brackets, the search direction (`ADVANCE/BACKUP`), `OVR` if overstrike mode is on, `SELECT` if a range is anchored, `*` if the buffer is modified, and a `col+N` indicator when the view is horizontally scrolled.
- The message line shows the last command's result (e.g. `Found: hello`, `RESET`, `Range deleted`).

### 3.3 Switching back to line mode

`Ctrl/Z` (control code 26) at any time in screen mode causes `handle_screen_key()` to return `true`, breaking the frame loop and returning to the calling `dispatch_command()` (`edt.cpp:3055-3057`). The editor immediately re-prompts with the `*` prompt.

## 4 Buffer model

EDT operates on a collection of named **buffers**. Each buffer is a `TextBuffer` instance (`edt.cpp:435-1161`) wrapping a `std::vector<std::string>` of lines. The map of buffers is held by the `Editor` class as `std::map<std::string, TextBuffer> buffers_` (`edt.cpp:3876`).

### 4.1 Canonical buffers

| Name               | Purpose                                                                                                                                        | Created when                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>MAIN</code>  | The file being edited; backed by <code>filename_</code>                                                                                        | At <code>Editor::run()</code><br>( <code>edt.cpp:1187-1193</code> ) |
| <code>PASTE</code> | Clipboard for CUT, APPEND, PASTE, REPLACE                                                                                                      | At <code>Editor::run()</code><br>( <code>edt.cpp:1191</code> )      |
| <i>user</i>        | Any name created by <code>FIND =name</code> , <code>COPY ... TO =name</code> , <code>MOVE ... TO =name</code> , or <code>DEFINE MACRO</code> . | First reference                                                     |

Buffer names are case-insensitive — `normalize_buffer_name()` (`edt.cpp:2190-2197`) strips any leading `=`, any surrounding double quotes, uppercases the result, and trims whitespace. An empty name becomes `MAIN`.

### 4.2 Modified flag

Each `TextBuffer` carries a `modified_` flag (`edt.cpp:1142`) that is flipped on by every mutating method (`insert_char`, `delete_line`, etc.) and cleared by `load()` (against the current filename) and `save()` / `write_file()` when the write target matches the buffer's filename (`edt.cpp:477`). `SHOW BUFFER` marks modified buffers with a trailing `*` (`edt.cpp:2264`); the screen-mode status line shows `*` next to the buffer name when the active buffer is dirty (`edt.cpp:3350`).

### 4.3 Line-number model

EDT line numbers are **1-based** and **decimal**. They are computed on the fly from the line's 0-based vector index — there is no persistent sequence number stored with each line. The `TYPE` / `DELETE` / etc. commands accept any of the following line references (`parse_line_ref()` at `edt.cpp:1361-1385`):

| Token                                                        | Meaning                                      |
|--------------------------------------------------------------|----------------------------------------------|
| <code>n</code> (positive integer)                            | Line <code>n</code> (clamped to buffer size) |
| <code>.</code> or <code>CURRENT</code>                       | The current cursor row                       |
| <code>BEGIN</code> , <code>FIRST</code> , <code>START</code> | Line 1 (first line)                          |
| <code>END</code> , <code>LAST</code>                         | The last line of the buffer                  |

These references compose into **ranges** via `parse_line_range()` (`edt.cpp:1390-1430`):

| Range syntax | Meaning                          |
|--------------|----------------------------------|
| WHOLE or ALL | Entire buffer                    |
| REST         | Cursor row through end of buffer |
| a THRU b     | Inclusive [a, b]                 |
| a:b          | Same as a THRU b                 |
| a-b          | Same as a THRU b (dash form)     |
| a            | Single-line range [a, a]         |

Endpoints are auto-swapped if reversed so the range always has `first <= last`. Signed offsets (`. + 5`, `END - 3`) are **not** implemented in the current build — see §13.

## 5 Line-mode commands — alphabetical reference

This section documents every verb dispatched by `dispatch_command()` (`edt.cpp:1739-1945`). The canonical list of recognised verbs is in `parse_command()` (`edt.cpp:1435-1590`). Each entry shows the abbreviated forms, the synopsis, the supported qualifiers, and an example. Where a VMS command is not implemented, see §13.

---

### ADVANCE

|             |                                                                                                                                                                                                                |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>ADVANCE</code> (alias <code>ADV</code> )                                                                                                                                                                 |
| Description | Sets the active search direction to forward. Subsequent <code>FNDNXT</code> operations scan from cursor toward end-of-buffer; <code>SUBSTITUTE</code> always advances.                                         |
| Range       | None                                                                                                                                                                                                           |
| Qualifiers  | None                                                                                                                                                                                                           |
| Source      | <code>edt.cpp:1829</code>                                                                                                                                                                                      |
| Note        | Stores the new direction in <code>direction_</code> . In line mode the response is <code>Direction: ADVANCE\n</code> ; in screen mode the status line flips from <code>BACKUP</code> to <code>ADVANCE</code> . |

Example:

```
* BACKUP
* FIND hello
* ADVANCE
* FNDNXT
```

### APPEND

|             |                                                                                                                                                                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>APPEND</code>                                                                                                                                                                                                               |
| Description | Append the current line, or the active <code>SELECT</code> range, to the <code>PASTE</code> buffer <b>without</b> clearing <code>PASTE</code> first. The source text is removed from the active buffer (it is moved, not copied). |
| Range       | Current line; or active <code>SELECT</code> range.                                                                                                                                                                                |
| Qualifiers  | None                                                                                                                                                                                                                              |
| Source      | <code>edt.cpp:1825</code> → <code>append_select()</code> at <code>edt.cpp:3652-3665</code>                                                                                                                                        |
| Note        | Repeated <code>APPEND</code> builds up an arbitrary clipboard. Use <code>FIND =PASTE</code> to inspect or edit the accumulated text.                                                                                              |

---

## BACKUP

|             |                                                |
|-------------|------------------------------------------------|
| Synopsis    | <code>BACKUP</code> (alias <code>BACK</code> ) |
| Description | Set the active search direction to backward.   |
| Source      | <code>edt.cpp:1830</code>                      |

Example: `BACKUP` followed by `FIND hello` scans backward from the cursor; `FNDNXT` after a `BACKUP` repeats backward.

---

## BOTTOM

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| Synopsis    | <code>BOTTOM</code>                                                               |
| Description | Move the cursor to the end of the last line.                                      |
| Source      | <code>edt.cpp:1822</code> → <code>go_bottom()</code> at <code>edt.cpp:3502</code> |

---

## C / CHANGE

|             |                                                                                                                                                                                                                                                                                                                  |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>CHANGE</code> (alias <code>C</code> ); <code>SET MODE CHANGE</code> ; <code>CHANGE ;nokeypad-cmds</code>                                                                                                                                                                                                   |
| Description | Switch from line mode into screen (keypad) mode. With <code>SET KEYPAD NOKEYPAD</code> , <code>CHANGE</code> enters the <b>nokeypad</b> command-line loop instead. Inline form <code>CHANGE ;cmds</code> runs a nokeypad command string without leaving line mode. Ctrl/Z returns from screen mode to line mode. |
| Source      | <code>CHANGE</code> / <code>SET MODE CHANGE</code> dispatch; <code>nokeypad_mode()</code>                                                                                                                                                                                                                        |

Nokeypad alphabet (no spaces): `C` / `W` / `L` / `EOL` / `BOL` / `PAGE`, `ADV` / `BACK`, `DC` / `DW` / `DL`, `SEL` / `CUT` / `PASTE` / `APPEND`, `TOP` / `BOTTOM`, `FIND`, `INSERT` ...`ESC`, `CHGC` / `CHNGCASE`, `CLSS` / `RESET`, `EX` / `EXIT` / `EXT`. See `HELP NOKEYPAD`.

---

## CHGL — Change to lowercase

|             |                                                                                                                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>CHGL</code> [ <code>range</code> ]                                                                                                                                                         |
| Description | Lower-case every character in the given range (or current line). If a <code>SELECT</code> range is active, that takes precedence; the <code>SELECT</code> is consumed when the command finishes. |
| Source      | <code>edt.cpp:1890</code> → <code>chg_case_command()</code> at <code>edt.cpp:1962-1990</code>                                                                                                    |

Example: `CHGL 1 THRU 10`

---

## CHGU — Change to uppercase

|             |                                                                                                |
|-------------|------------------------------------------------------------------------------------------------|
| Synopsis    | <code>CHGU [range]</code>                                                                      |
| Description | Upper-case every character in the given range (or current line). Mirror of <code>CHGL</code> . |
| Source      | <code>edt.cpp:1891</code>                                                                      |

## CLEAR

|             |                                                                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>CLEAR [name]</code>                                                                                                                                               |
| Description | Truncate MAIN or PASTE to a single empty line; <b>delete</b> any user-defined buffer or macro of the given name. If no name is supplied, the current buffer is cleared. |
| Source      | <code>edt.cpp:1875</code> → <code>clear_buffer_command()</code> at <code>edt.cpp:2283-2299</code>                                                                       |

Example: `CLEAR TEMP` removes the user buffer `TEMP`; `CLEAR PASTE` empties the clipboard.

## COPY

|             |                                                                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>COPY range TO =buffer</code>                                                                                                                                                          |
| Description | Copy the lines in <code>range</code> from the active buffer onto the <b>end</b> of <code>=buffer</code> . The source range is left untouched. Use <code>MOVE</code> if you want it removed. |
| Source      | <code>edt.cpp:1876</code> → <code>copy_move_command()</code> at <code>edt.cpp:2309-2334</code>                                                                                              |
| Note        | The destination buffer is created on first use.                                                                                                                                             |

Examples:

```
* COPY 10 THRU 20 TO =TEMP
* COPY WHOLE TO =BACKUP
* COPY . TO =LINE_LOG
```

## CUT

|             |                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>CUT</code>                                                                                                   |
| Description | Move the current line (or active SELECT range) into the PASTE buffer, <b>overwriting</b> any prior PASTE contents. |
| Source      | <code>edt.cpp:1824</code> → <code>cut_select()</code> at <code>edt.cpp:3638-3650</code>                            |

## DEFINE KEY

|             |                                                                                                                                                                                                |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>DEFINE KEY [GOLD] [CONTROL] key AS "command-string"</code>                                                                                                                               |
| Description | Bind a key (with optional GOLD and/or CONTROL prefixes) to a command string. The command string is replayed through <code>execute_command()</code> whenever the key is pressed in screen mode. |
| Source      | <code>edt.cpp:1926-1930</code> → <code>define_key_command()</code> at <code>edt.cpp:2809-2835</code>                                                                                           |

The recognised key tokens are `PF1/F1`, `PF2/F2`, `PF3/F3`, `PF4/F4`, `DELETE`, and any single ASCII character (`normalize_key_spec()` at `edt.cpp:2841-2875`). User-defined keys take precedence over the hard-coded bindings (`try_defined_key()` at `edt.cpp:2903-2911`). See §10 for examples.

---

## DEFINE MACRO

|             |                                                                                                                                                                                                |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>DEFINE MACRO name AS "cmd; cmd; ..."</code> or <code>DEFINE MACRO name</code> (interactive)                                                                                              |
| Description | Register a named sequence of commands. The body may be supplied inline (separated by <code>;</code> or <code>\n</code> ) or read interactively until a blank line or <code>END DEFINE</code> . |
| Source      | <code>edt.cpp:1887</code> → <code>define_macro_command()</code> at <code>edt.cpp:2695-2740</code>                                                                                              |

Restrictions:

- The name must not be `MAIN` or `PASTE`.
  - Macro names occupy the same namespace as user buffers — defining a macro creates a buffer with the same name.
  - Macros are run with a recursion guard (`running_macros_`); a macro that calls itself (or a mutually-recursive cycle) is rejected with `?Recursive macro invocation`.
- 

## DELETE

|             |                                                                                                                                                                                                                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>DELETE [range]</code> (alias <code>D</code> )                                                                                                                                                                                                                                                 |
| Description | Delete the specified range, or the current line if no range is given. The removed text is captured in the line undelete buffer ( <code>delete_line_buffer_</code> ) and can be restored with <code>UND L</code> . If a SELECT range is active when called without arguments, that range is deleted. |
| Source      | <code>edt.cpp:1790-1805</code> , <code>edt.cpp:1472</code> , <code>1540</code>                                                                                                                                                                                                                      |

Examples:

```
* DELETE
* DELETE 5
* DELETE 10 THRU 20
* DELETE REST
* DELETE WHOLE
```

---

## DEL C / DC / DELETE CHARACTER

|             |                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>DEL C</code> (also <code>DC</code> , <code>DELETE CHARACTER</code> )                                                                  |
| Description | Forward-delete a single character at the cursor. Stashes the deleted character in <code>delete_char_buffer_</code> for <code>UND C</code> . |
| Source      | <code>edt.cpp:1488, 1833</code>                                                                                                             |

## DEL EOL

|             |                                                                                           |
|-------------|-------------------------------------------------------------------------------------------|
| Synopsis    | <code>DEL EOL</code> (also <code>DELETE EOL</code> , <code>DELETE TO END OF LINE</code> ) |
| Description | Delete from cursor to end of line; the removed text is placed in PASTE.                   |
| Source      | <code>edt.cpp:1492, 1837</code>                                                           |

## DEL L / DL / DELETE LINE

|             |                                                                                              |
|-------------|----------------------------------------------------------------------------------------------|
| Synopsis    | <code>DEL L</code> (also <code>DL</code> , <code>DELETE LINE</code> )                        |
| Description | Delete the current line, or the active SELECT range, into <code>delete_line_buffer_</code> . |
| Source      | <code>edt.cpp:1486, 1831</code>                                                              |

## DEL W / DW / DELETE WORD

|             |                                                                                                                                                                                       |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>DEL W</code> (also <code>DW</code> , <code>DELETE WORD</code> )                                                                                                                 |
| Description | Delete the next word forward, stashing it in <code>delete_word_buffer_</code> for <code>UND W</code> . Word boundaries respect the user's <code>SET ENTITY WORD</code> configuration. |
| Source      | <code>edt.cpp:1490, 1835</code>                                                                                                                                                       |

## EXIT

|             |                                                                                                                                                                                                                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>EXIT</code> (alias <code>EX</code> )                                                                                                                                                                                                                                  |
| Description | Save the MAIN buffer to its backing file (or <code>-output=</code> path) and terminate the process. Exit code is <code>0</code> on success, <code>2</code> if the write fails. On a successful save, the session journal ( <code>file.jou</code> ) is <b>deleted</b> (§11). |
| Source      | <code>save_and_exit_now()</code> / <code>delete_journal_file()</code>                                                                                                                                                                                                       |

## FILL

|             |                                                                                                                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>FILL</code>                                                                                                                                                                                   |
| Description | Reflow the paragraph at the cursor (or the active SELECT range) to fit the terminal width. Paragraph boundaries are blank lines. Wrap width is <code>terminal_cols - 1</code> , with a floor of 20. |
| Source      | <code>edt.cpp:1883</code> → <code>fill_command()</code> at <code>edt.cpp:2492-2570</code>                                                                                                           |
| Note        | Outside screen mode the wrap width falls back to 79.                                                                                                                                                |

## FIND

|             |                                                                                                                                                                                                                                                                                                                                          |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>FIND text</code> or <code>FIND =buffer</code> (alias <code>F</code> ); optional <code>/BRIEF[:n]</code>                                                                                                                                                                                                                            |
| Description | Set the active search string ( <code>last_search_</code> ) and locate the next match in the forward direction. With <code>=buffer</code> , switch the active buffer to the named one. <code>/BRIEF</code> truncates the typed match line (same as <code>TYPE</code> ). Honour <code>SET SEARCH WORD/BEGIN/END</code> constraints (§8.6). |
| Source      | <code>edt.cpp:1545-1550, 1810-1820</code> → <code>find_buffer()</code> at <code>edt.cpp:2222-2238</code>                                                                                                                                                                                                                                 |

Examples:

```
* FIND hello
* FIND "Hello World"
* FIND =PASTE
* FIND =TEMP
```

The needle may use the EDT wildcard set (`*`, `?`, `[abc]`, `[^abc]`, `^`, `$`, `\\x`) — see §8.

## FNDNXT / FIND NEXT

|             |                                                                                                                               |
|-------------|-------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>FNDNXT</code> or <code>FIND NEXT</code>                                                                                 |
| Description | Repeat the last <code>FIND</code> in the active direction. Wraps from end to start (or start to end on <code>BACKUP</code> ). |
| Source      | <code>edt.cpp:1478, 1821</code> → <code>find_next()</code> at <code>edt.cpp:3759-3790</code>                                  |

## GOTO-LINE (synthetic)

|             |                                                                                                                                                                                                                                    |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | Bare number, e.g. <code>42</code>                                                                                                                                                                                                  |
| Description | Move the cursor to line <code>n</code> and type the line. Synthesised by <code>parse_command()</code> from a bare numeric prefix ( <code>edt.cpp:1452-1456</code> ); the dispatcher handles it at <code>edt.cpp:1748-1765</code> . |

## HELP

|             |                                                                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>HELP [topic]</code> (alias <code>H</code> )                                                                                                                          |
| Description | Print line-mode help. Topics include <code>SET</code> , <code>SHOW</code> , <code>CHANGE</code> , <code>KEYPAD</code> , <code>NOKEYPAD</code> , and <code>VERSION</code> . |
| Source      | <code>print_line_help()</code>                                                                                                                                             |

## INCLUDE

|             |                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>INCLUDE filename</code>                                                                                      |
| Description | Insert the contents of <code>filename</code> at the current cursor row. CR/LF line endings are normalised on read. |
| Source      | <code>edt.cpp:1878</code> → <code>include_file_command()</code> at <code>edt.cpp:2338-2353</code>                  |

## INSERT

|             |                                                                                                                                                                                                                                                                                                                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>INSERT text</code> (alias <code>I</code> ); bare <code>INSERT</code> for multiline                                                                                                                                                                                                                                                                                    |
| Description | Insert literal <code>text</code> at the cursor position. Surrounding double quotes are stripped; embedded newlines trigger <code>split_line</code> . Bare <b>INSERT</b> (no argument) enters multiline insert: successive lines are read with a <code>_</code> continuation prompt until a terminator line that is Ctrl/Z alone, <code>^Z</code> , <code>.</code> , or EOF. |
| Source      | <code>INSERT</code> dispatch → <code>paste_text()</code> / <code>insert_multiline()</code>                                                                                                                                                                                                                                                                                  |

Examples:

```
* INSERT Hello, world
* INSERT "indented text  "
* INSERT
first line
_second line
_.
```

## MOVE

|             |                                                                                                                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>MOVE range TO =buffer</code>                                                                                                                                          |
| Description | Append <code>range</code> onto <code>=buffer</code> and delete the lines from the source. Same machinery as <code>COPY</code> except the source range is removed afterward. |
| Source      | <code>edt.cpp:1877</code> → <code>copy_move_command()</code> ( <code>move=true</code> )                                                                                     |

## NEXT

|             |                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>NEXT</code>                                                                                                          |
| Description | Save the current MAIN buffer to disk and load the next file from the argv file list. Cursor/view/selection state is reset. |
| Source      | <code>edt.cpp:1893, 2017-2035</code>                                                                                       |

---

## PASTE

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| Synopsis    | <code>PASTE</code>                                                            |
| Description | Insert the contents of the PASTE buffer at the cursor without clearing PASTE. |
| Source      | <code>edt.cpp:1823, 3667-3670</code>                                          |

---

## PREVIOUS

|             |                                                                     |
|-------------|---------------------------------------------------------------------|
| Synopsis    | <code>PREVIOUS</code> (alias <code>PREV</code> )                    |
| Description | Mirror of <code>NEXT</code> — moves backward through the file list. |
| Source      | <code>edt.cpp:1894, 2037-2055</code>                                |

---

## PRINT

|             |                                                                                                                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>PRINT [range]</code>                                                                                                                                                       |
| Description | Write the buffer (or the given range) to a temp file and shell out to <code>lpr</code> or <code>lp</code> . On Windows the <code>ShellExecute("print", ...)</code> verb is used. |
| Source      | <code>edt.cpp:1884</code> → <code>print_command()</code> at <code>edt.cpp:2581-2642</code>                                                                                       |

---

## QUIT

|             |                                                                                                                                                                                                            |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>QUIT</code> (alias <code>Q</code> )                                                                                                                                                                  |
| Description | Discard all changes and exit with status <code>0</code> (VMS-compatible; OpenVMS leaves <code>\$STATUS = SS\$_NORMAL</code> after QUIT). Prints <code>Changes discarded</code> if any buffer was modified. |
| Source      | <code>edt.cpp:1470, 1776, 3389-3399</code>                                                                                                                                                                 |

---

## REDO

|             |                                                                                                                                                      |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>REDO</code>                                                                                                                                    |
| Description | Re-apply the most recently undone edit. Pops one entry from <code>redo_stack_</code> and snapshots the current state onto <code>undo_stack_</code> . |
| Source      | <code>edt.cpp:1474, 1766, 1681-1700</code>                                                                                                           |

---

## REPLACE

|             |                                                                                                      |
|-------------|------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>REPLACE</code>                                                                                 |
| Description | Replace the current line (or active SELECT range) with the contents of PASTE. PASTE is not consumed. |
| Source      | <code>edt.cpp:1826, 3685-3705</code>                                                                 |

---

## RESET

|             |                                                                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>RESET</code>                                                                                                                                                   |
| Description | Reset the search direction to ADVANCE, clear any pending GOLD modifier, and cancel an active SELECT range. Does <b>not</b> affect the cursor or the buffer contents. |
| Source      | <code>edt.cpp:1828, 3569-3574</code>                                                                                                                                 |

---

## RESEQUENCE

|             |                                                                                                                                                                                                                                                                                                                  |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>RESEQUENCE</code> (alias <code>RESEQ</code> ); <code>RESEQUENCE /SEQUENCE=start:incr</code>                                                                                                                                                                                                                |
| Description | Rewrite an 8-column sequence prefix ( <code>NNNNN.00</code> ) on every line of the active buffer. Default start/increment are <code>100</code> . <code>/SEQUENCE=start:incr</code> overrides both. Existing text at columns 1–8 is replaced; shorter lines are padded. <code>/DUPLICATE</code> is soft-accepted. |
| Source      | <code>resequence_command()</code>                                                                                                                                                                                                                                                                                |

Example:

```
* RESEQUENCE /SEQUENCE=100:100
```

---

## SELECT

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>SELECT</code> (alias <code>SEL</code> )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Description | Drop a character-position <b>mark</b> at the cursor ( <code>TextBuffer::set_mark</code> at <code>edt.cpp:1050</code> ). The select range is the half-open character span between the mark and the cursor, in document order — exclusive of the cursor — matching OpenVMS EDT V9.2-3. A second <code>SELECT</code> while one is active is rejected. <code>CUT</code> , <code>APPEND</code> , <code>REPLACE</code> , <code>CHGL/CHGU</code> , and <code>FILL</code> consume the range; <code>DEL L</code> ignores it. <code>RESET</code> or <code>GOLD+</code> . clears the mark. Screen highlighting is character-granular ( <code>active_select_span / A_REVERSE</code> ). |
| Source      | <code>edt.cpp:3527, 3549, 1050, 1086</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

## SET ...

See §6 for the complete `SET/SHOW` parameter list. The verb-level entries the parser recognises are listed below for completeness — every one has a matching `SHOW` form:

```
SET NUMBERS / NONUMBERS      SET SEARCH EXACT / GENERAL / WORD / ...
SET MODE CHANGE / LINE       SET OVERSTRIKE / INSERT
SET QUIET / NOQUIET          SET VERIFY / NOVERIFY
SET WRAP / NOWRAP            SET TAB n / NOTAB
SET PROMPT "... "            SET ENTITY WORD "delims"
SET KEYPAD EDT / NOKEYPAD / WPS SET CURSOR top bottom
```

## SHOW ...

See §6. Implemented: `SHOW BUFFER`, `SHOW FILES`, `SHOW MODE`, `SHOW VERSION`, `SHOW NUMBERS`, `SHOW SEARCH`, `SHOW PROMPT`, `SHOW KEY [pattern]`, `SHOW ENTITY / SHOW WORD`, `SHOW QUIET`, `SHOW VERIFY`, `SHOW WRAP`, `SHOW TAB`, `SHOW MACRO [name]`, plus the `SET/SHOW` companions for `AUTOREPEAT`, `CASE`, `COMMAND`, `CURSOR`, `FNF`, `HELP`, `KEYPAD`, `LINES`, `PARAGRAPH`, `REPEAT`, `SCREEN`, `SUMMARY`, `TERMINAL`, `TEXT`, `TRUNCATE`.

`VERSION` is an alias of `SHOW VERSION`. Output matches:

```
V1.02-0      COPYRIGHT (c) Sector7 USA LLC 2026
Release date: 2026-07-15
```

Format: `V{major}.{minor:02d}-{edit}` plus copyright and `Release date: YYYY-MM-DD`. Bump `EDT_VERSION_*` and `EDT_RELEASE_DATE` in `edt.cpp` on every public release.

## SUBSTITUTE

|             |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>SUBSTITUTE [replacement] (alias SUBS); SUBSTITUTE/old/new/; SUBSTITUTE NEXT; SUBSTITUTE ALL [replacement]</code>                                                                                                                                                                                                                                                                                              |
| Description | Replace the next occurrence of the active search string with <code>replacement</code> . If <code>replacement</code> is omitted, the PASTE buffer is used. The three-delimiter form sets the search string to <code>old</code> and replaces with <code>new</code> . After replacement the cursor advances to the next match unless <code>/STAY</code> was used. <code>SUBSTITUTE ALL</code> walks the entire buffer. |
| Qualifiers  | <code>/QUERY</code> , <code>/GLOBAL</code> , <code>/G</code> , <code>/NOTYPE</code> , <code>/SAVE</code> , <code>/STAY</code>                                                                                                                                                                                                                                                                                       |

Examples:

```
* SUBSTITUTE done
* SUBSTITUTE/old/new/
* SUBSTITUTE/QUERY done
* SUBSTITUTE/GLOBAL/QUERY done
* SUBSTITUTE ALL done
```

## TAB

|             |                                                                                                                                         |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>TAB</code>                                                                                                                        |
| Description | Insert spaces from the cursor to the next <code>SET TAB</code> stop (default width 8). Same spacing rule as the Tab key in screen mode. |
| Source      | <code>insert_tab_at_cursor()</code>                                                                                                     |

## TAB ADJUST

|             |                                                                                                              |
|-------------|--------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>TAB ADJUST [range]</code>                                                                              |
| Description | Normalise leading whitespace on the current line or range to the active <code>SET TAB</code> width (spaces). |

`SET TAB n` and `SHOW TAB` remain available (§6); the TAB key in screen mode expands to spaces at the configured tab stops.

## TOP

|             |                                             |
|-------------|---------------------------------------------|
| Synopsis    | <code>TOP</code>                            |
| Description | Move the cursor to the beginning of line 1. |
| Source      | <code>edt.cpp:1480, 1821, 3500</code>       |

## TYPE / PRINT (display)

|             |                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>TYPE [range]</code> (alias <code>T</code> ); optional <code>/BRIEF</code> or <code>/BRIEF:n</code>                                                                                                                                                                                                                                                                                |
| Description | Display the current line, or the lines in <code>range</code> , on stdout in the form <code>&lt;n&gt; &lt;text&gt; .</code> <code>[EOB]</code> is appended if the range reaches end-of-buffer. <code>/BRIEF</code> truncates each line's text to 40 columns (or <code>n</code> with <code>/BRIEF:n</code> ). Ctrl/C during long TYPE/FIND/SUBSTITUTE loops prints <code>*Abort*</code> . |
| Source      | <code>type_range()</code> / qualifier parse                                                                                                                                                                                                                                                                                                                                             |

Examples:

```
* TYPE
* TYPE 10
* TYPE 1 THRU 20
* TYPE 1:5
* TYPE REST
* TYPE WHOLE
* TYPE ALL
* TYPE /BRIEF:20 1 THRU 5
```

## UNDO

|             |                                                                                                                                                                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>UNDO</code> (alias <code>U</code> )                                                                                                                                                                                                                               |
| Description | Pop the most recent snapshot from <code>undo_stack_</code> and restore it. Up to 100 undo levels are retained ( <code>max_undo_</code> at <code>edt.cpp:3908</code> ). The current state is pushed onto <code>redo_stack_</code> so <code>REDO</code> can step forward. |
| Source      | <code>edt.cpp:1473, 1765, 1660-1678</code>                                                                                                                                                                                                                              |

## UND L / UNDELETE LINE

|             |                                                                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | <code>UND L</code> (also <code>UNDELETE LINE</code> )                                                                             |
| Description | Re-insert the line(s) most recently removed by <code>DEL L</code> / PF4. The text is stored in <code>delete_line_buffer_</code> . |
| Source      | <code>edt.cpp:1487, 1832, 3615-3622</code>                                                                                        |

## UND C / UNDELETE CHARACTER

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| Synopsis    | UND C (also UNDELETE CHARACTER)                                               |
| Description | Re-insert the character most recently deleted by DEL C , backspace, or KP , . |
| Source      | edt.cpp:1489, 1834                                                            |

---

## UND W / UNDELETE WORD

|             |                                                            |
|-------------|------------------------------------------------------------|
| Synopsis    | UND W (also UNDELETE WORD)                                 |
| Description | Re-insert the word most recently deleted by DEL W / KP - . |
| Source      | edt.cpp:1491, 1836                                         |

---

## WRITE

|             |                                                                                                                                                                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synopsis    | WRITE filename [range] [BUFFER name]                                                                                                                                                                                                                                                   |
| Description | Write the active buffer (or the named buffer, with BUFFER name) to filename . An optional line range (n , a:b , a THRU b , WHOLE / ALL ) writes only those lines. The active buffer's modified_ flag is <b>not</b> cleared unless filename matches the buffer's recorded backing file. |
| Source      | write_command()                                                                                                                                                                                                                                                                        |

Examples:

```
* WRITE backup.txt
* WRITE out.txt 1 THRU 10
* WRITE slice.txt 1:5 BUFFER TEMP
* WRITE out.txt BUFFER TEMP
```

---

## RANGE

The standalone RANGE command from VMS EDT (which prints the cursor and "implied" range used by the next command) is **not** present in this build. Range tokens are documented in §4 and apply to TYPE, DELETE, COPY, MOVE, CHGL, CHGU, PRINT.

---

## RETAIN / STORE TEXT

These VMS commands are **not** implemented. The closest equivalents in this build are COPY range TO =name and WRITE file BUFFER name .

---

## Command qualifiers (slash flags)

Qualifiers stripped by `extract_qualifiers()` (`edt.cpp:1318-1342`):

| Qualifier                                    | Meaning                                  | Status                                                          |
|----------------------------------------------|------------------------------------------|-----------------------------------------------------------------|
| <code>/QUERY</code>                          | Confirm each affected line/match         | Implemented for SUBSTITUTE (§5/<br>SUBSTITUTE)                  |
| <code>/GLOBAL</code> or <code>/G</code>      | Apply to every match in the buffer       | Implemented for SUBSTITUTE                                      |
| <code>/NOTYPE</code>                         | Suppress this command's stdout           | Implemented (silences <code>report</code> for the call)         |
| <code>/BRIEF</code> or <code>/BRIEF:n</code> | Abbreviated TYPE/FIND output             | Implemented (default 40 columns; <code>:n</code> sets<br>width) |
| <code>/SAVE</code>                           | Save buffer after the command            | Implemented ( <code>do_save</code> after dispatch)              |
| <code>/STAY</code>                           | Do not advance after FIND/<br>SUBSTITUTE | Implemented                                                     |
| <code>/SEQUENCE=start:incr</code>            | RESEQUENCE numbering                     | Implemented                                                     |

Unknown qualifiers are passed through unchanged so the dispatcher can report them. See §13 for the qualifiers VMS supports that are missing.

## 6 SET and SHOW parameter reference

Parameters are stored on `Editor` and toggled by `SET-*` / `SHOW-*` verbs in `dispatch_command()`. Defaults come from member initialisers.

| Parameter                | SET form                                                  | SHOW form                    | Default             | Effect                                          |
|--------------------------|-----------------------------------------------------------|------------------------------|---------------------|-------------------------------------------------|
| <b>NUMBERS</b>           | SET NUMBERS /<br>NONUMBERS                                | SHOW NUMBERS                 | OFF                 | Line-number prefix in screen mode               |
| <b>SEARCH</b>            | SET SEARCH EXACT /<br>GENERAL (+ BEGIN/<br>END/WILD/WORD) | SHOW SEARCH                  | EXACT               | FIND case / flags                               |
| <b>MODE</b>              | SET MODE CHANGE /<br>LINE                                 | SHOW MODE                    | LINE                | Enter screen mode or<br>acknowledge line        |
| <b>OVERSTRIKE</b>        | SET OVERSTRIKE /<br>INSERT ...                            | status OVR                   | INSERT              | Overwrite vs insert                             |
| <b>BUFFER</b>            | via FIND =name                                            | SHOW<br>BUFFER(S)            | MAIN                | Named buffers                                   |
| <b>QUIET</b>             | SET QUIET /<br>NOQUIET                                    | SHOW QUIET                   | OFF                 | Suppress banners                                |
| <b>VERIFY</b>            | SET VERIFY /<br>NOVERIFY                                  | SHOW VERIFY                  | OFF                 | Echo macros                                     |
| <b>WRAP</b>              | SET WRAP / NOWRAP                                         | SHOW WRAP                    | OFF                 | Soft-wrap on insert when width<br>known         |
| <b>TAB</b>               | SET TAB n / NOTAB                                         | SHOW TAB                     | 8                   | Tab stops                                       |
| <b>PROMPT</b>            | SET PROMPT "..."                                          | SHOW PROMPT                  | *                   | Line-mode prompt                                |
| <b>ENTITY /<br/>WORD</b> | SET ENTITY WORD<br>"..." / SET WORD ...                   | SHOW<br>ENTITY / WORD        | (default<br>delims) | Word motion delimiters                          |
| <b>FILES</b>             | —                                                         | SHOW FILES                   | —                   | Input / output paths                            |
| <b>VERSION</b>           | —                                                         | SHOW<br>VERSION /<br>VERSION | —                   | Vmajor.minor-edit +<br>release date             |
| <b>AUTOREPEAT</b>        | SET AUTOREPEAT /<br>NO...                                 | SHOW<br>AUTOREPEAT           | ON                  | (stored)                                        |
| <b>CASE</b>              | SET CASE NONE /<br>LOWER / UPPER                          | SHOW CASE                    | NONE                | Transform INSERT letters                        |
| <b>COMMAND</b>           | SET COMMAND file                                          | SHOW COMMAND                 | (none)              | Extra command file path                         |
| <b>CURSOR</b>            | SET CURSOR top<br>bottom                                  | SHOW CURSOR                  | 0 0                 | Soft scroll margins (applied in<br>adjust_view) |
| <b>FNF</b>               | SET FNF / NOFNF                                           | SHOW FNF                     | ON                  | "Input file does not exist" banner              |
| <b>HELP</b>              | SET HELP name                                             | SHOW HELP                    | builtin             | Help file name                                  |

| Parameter        | SET form                                | SHOW form         | Default              | Effect                                                                                   |
|------------------|-----------------------------------------|-------------------|----------------------|------------------------------------------------------------------------------------------|
| <b>KEYPAD</b>    | SET KEYPAD EDT /<br>NOKEYPAD / WPS / PC | SHOW KEYPAD       | EDT                  | Keypad family; NOKEYPAD enters nokeypad language; PC maps keypad / * - to PF2/3/4 (§7.3) |
| <b>LINES</b>     | SET LINES n                             | SHOW LINES        | 0                    | Height override (0 = terminal)                                                           |
| <b>PARAGRAPH</b> | SET PARAGRAPH<br>WPS / NOWPS            | SHOW<br>PARAGRAPH | NOWPS                | Paragraph style flag                                                                     |
| <b>REPEAT</b>    | SET REPEAT /<br>NOREPEAT                | SHOW REPEAT       | ON                   | GOLD digit-repeat                                                                        |
| <b>SCREEN</b>    | SET SCREEN n                            | SHOW SCREEN       | 0                    | Width for FILL/wrap (0 = auto)                                                           |
| <b>SUMMARY</b>   | SET SUMMARY /<br>NOSUMMARY              | SHOW SUMMARY      | ON                   | Exit summary flag                                                                        |
| <b>TERMINAL</b>  | SET TERMINAL type                       | SHOW<br>TERMINAL  | (host)               | Reported type string                                                                     |
| <b>TEXT</b>      | SET TEXT<br>FORMFEED / END ...          | SHOW TEXT         | form-feed /<br>[EOB] | Display strings                                                                          |
| <b>TRUNCATE</b>  | SET TRUNCATE /<br>NOTRUNCATE            | SHOW<br>TRUNCATE  | ON                   | Truncate flag                                                                            |
| <b>KEY</b>       | via DEFINE KEY                          | SHOW KEY<br>[pat] | —                    | User key bindings                                                                        |

## 6.1 Still incomplete vs OpenVMS

- Rare **WPS-only** nokeypad entities may differ; SET KEYPAD WPS aliases to EDT with a notice.
- Some VMS punctuation / Files-11-specific settings are host-N/A.
- **RMS** record formats and ;n versions remain deferred (§13.6).

## 7 Keypad reference (screen mode)

In screen mode every keystroke flows through `handle_screen_key()` (`edt.cpp:3045-3125`). If GOLD was the previous key (`gold_pending_ == true`) the dispatch instead goes through `handle_gold_key()` (`edt.cpp:3128-3166`). User-defined keys (§10) are checked first; if none match, the hard-coded tables below apply.

### 7.1 Keypad layouts (VT100 / VT220 and PC)

EDT's screen mode is driven by the numeric keypad. GOLD (PF1) is a *prefix*: press it, then a keypad key, to get the key's GOLD-shifted function.

Each key below is shown on three lines — its **KeyID**, its **normal function**, and its **GOLD-shifted function** (prefixed `g:`), the way the classic DEC EDT keypad card did it. A few GOLD names are abbreviated to fit the cell; the full names are in §7.2 and §7.3.

#### 7.1.1 VT100 / VT220 keypad

|             |          |          |           |  |
|-------------|----------|----------|-----------|--|
| PF1         | PF2      | PF3      | PF4       |  |
| GOLD        | HELP     | FNDNXT   | DEL L     |  |
| g: (GOLD)   | g:BOTTOM | g:FIND   | g:UND L   |  |
| 7           | 8        | 9        | -         |  |
| PAGE        | SECT     | APPEND   | DEL W     |  |
| g:COMMND    | g:FILL   | g:REPLC  | g:UND W   |  |
| 4           | 5        | 6        | ,         |  |
| ADVANCE     | BACKUP   | CUT      | DEL C     |  |
| g:BOTTOM    | g:TOP    | g:PASTE  | g:UND C   |  |
| 1           | 2        | 3        | ENTER     |  |
| WORD        | EOL      | CHAR     | (return)  |  |
| g:CHGCAS    | g:DELEOL | g:SPCINS | g: (same) |  |
| 0           | .        |          |           |  |
| LINE        | SELECT   |          |           |  |
| g:OPEN LINE | g:RESET  |          |           |  |

Abbreviations: COMMND = COMMAND, REPLC = REPLACE, CHGCAS = CHNGCASE, DELEOL = DEL EOL, SPCINS = SPECINS.

#### 7.1.2 PC keyboard (numeric keypad)

A PC numeric keypad has a different top row (Num Lock, `/`, `*`, `-`) than a DEC keypad (PF1–PF4). The usual convention — and the one PuTTY uses in application-keypad mode — maps the PC top row onto PF1–PF4:

| PC key   | Acts as  | Normal function    | GOLD function            |
|----------|----------|--------------------|--------------------------|
| Num Lock | PF1      | GOLD (prefix)      | re-arm GOLD              |
| /        | PF2      | HELP               | BOTTOM                   |
| *        | PF3      | FNDNXT             | FIND                     |
| -        | PF4      | DEL L              | UND L                    |
| 7 8 9    | KP7/8/9  | PAGE/SECT/APPEND   | COMMAND/FILL/REPLACE     |
| +        | KP-      | DEL W              | UND W                    |
| 4 5 6    | KP4/5/6  | ADVANCE/BACKUP/CUT | BOTTOM/TOP/PASTE         |
| 1 2 3    | KP1/2/3  | WORD/EOL/CHAR      | CHNGCASE/DEL EOL/SPECINS |
| 0        | KP0      | LINE               | OPEN LINE                |
| . (Del)  | KP.      | SELECT             | RESET                    |
| Enter    | KP Enter | newline            | newline                  |

Notes for PC keyboards:

- These mappings require the terminal to be in **application-keypad mode** (EDT requests it on entering screen mode). In *numeric* keypad mode the keys just type digits.
- **Num Lock → PF1 (GOLD)**: most emulators send PF1 from Num Lock only in application-keypad mode. In macOS Terminal.app this is not remappable — use PuTTY, iTerm2, or another VT100-faithful emulator. See also the `SET KEYPAD PC` option (§7.3) which lets EDT itself treat the keypad `/ * -` keys as PF2/PF3/PF4.
- The PC `+` key has no DEC equivalent; emulators commonly send it as the keypad `-` (DEL W).

## 7.2 PF-key bindings

| Key      | Normal                                | GOLD-shifted                                      | Source                                                    |
|----------|---------------------------------------|---------------------------------------------------|-----------------------------------------------------------|
| PF1 / F1 | GOLD prefix (next key is shifted)     | Re-arms GOLD                                      | <code>edt.cpp:3058-3061,</code><br><code>3134-3137</code> |
| PF2 / F2 | HELP screen (full keypad cheat sheet) | BOTTOM (OpenVMS-aligned)                          | <code>edt.cpp:3062-3064,</code><br><code>3138-3141</code> |
| PF3 / F3 | FNDNXT (repeat last search)           | FIND prompt — read a new search string and search | <code>edt.cpp:3065-3067,</code><br><code>3141-3143</code> |
| PF4 / F4 | Delete current line (DEL L)           | UND L — undelete the most recently deleted line   | <code>edt.cpp:3068-3070,</code><br><code>3144-3146</code> |

## 7.3 Numeric keypad bindings

| Key         | Normal command                                       | GOLD-shifted                                          | Source                          |
|-------------|------------------------------------------------------|-------------------------------------------------------|---------------------------------|
| KP0         | LINE — open line (split at cursor)                   | OPEN LINE (same — insert blank line below)            | <code>edt.cpp:3184, 3243</code> |
| KP1         | WORD — word motion in active direction               | CHNGCASE — flip case of char at cursor and step right | <code>edt.cpp:3185, 3244</code> |
| KP2         | EOL — move to end of line                            | DEL EOL — delete from cursor to EOL into PASTE        | <code>edt.cpp:3186, 3245</code> |
| KP3         | CHAR — single-character motion                       | SPECINS — prompt for literal text to insert           | <code>edt.cpp:3187, 3246</code> |
| KP4         | ADVANCE — set direction forward                      | BOTTOM                                                | <code>edt.cpp:3188, 3247</code> |
| KP5         | BACKUP — set direction backward                      | TOP                                                   | <code>edt.cpp:3189, 3248</code> |
| KP6         | CUT — move line/range into PASTE                     | PASTE — insert PASTE at cursor                        | <code>edt.cpp:3190, 3249</code> |
| KP7         | PAGE — page down                                     | COMMAND — open line-mode command prompt               | <code>edt.cpp:3191, 3250</code> |
| KP8         | SECT — 16-line section motion                        | FILL — reflow paragraph at cursor                     | <code>edt.cpp:3192, 3251</code> |
| KP9         | APPEND — move line/range into PASTE without clearing | REPLACE — replace current line/range with PASTE       | <code>edt.cpp:3193, 3252</code> |
| KP-         | DEL W — delete next word forward                     | UND W — undelete word                                 | <code>edt.cpp:3195, 3254</code> |
| KP,         | DEL C — forward-delete one character                 | UND C — undelete character                            | <code>edt.cpp:3196, 3255</code> |
| KP.         | SELECT — anchor a SELECT range                       | RESET — clear SELECT, GOLD, reset direction           | <code>edt.cpp:3194, 3253</code> |
| KP<br>Enter | Newline (split line, advance row)                    | Newline (same — no distinct GOLD form)                | <code>edt.cpp:3197-3201</code>  |

**SET KEYPAD PC** — On a PC numeric keypad the top row is `/ * -` rather than PF2/PF3/PF4. `SET KEYPAD PC` (also settable in `EDT.INI`) makes EDT treat those keys as **PF2 (HELP)**, **PF3 (FNDNXT)** and **PF4 (DEL L)** in screen mode, matching PuTTY's application-keypad convention. DEL W then moves to the PC `+` key. The default (`SET KEYPAD EDT`) keeps the strict DEC mapping where keypad `-` is DEL W. Num Lock still needs to send PF1 from the emulator to act as GOLD (see §7.1.2).

## 7.4 Control-key bindings

| Key    | Action                                                                     | Source |
|--------|----------------------------------------------------------------------------|--------|
| Ctrl/A | Toggle insert/overstrike mode                                              |        |
| Ctrl/B | Move cursor to beginning of line                                           |        |
| Ctrl/C | Clear GOLD / select (screen); <code>*Abort*</code> on long line-mode loops |        |
| Ctrl/D | Toggle ADVANCE / BACKUP direction                                          |        |
| Ctrl/E | Move cursor to end of line                                                 |        |
| Ctrl/F | FNDNXT                                                                     |        |
| Ctrl/J | LINEFEED — delete word left                                                |        |
| Ctrl/K | Interactive DEFINE KEY (press key, then definition)                        |        |
| Ctrl/L | Refresh screen                                                             |        |
| Ctrl/R | Refresh screen                                                             |        |
| Ctrl/T | Toggle case of character at cursor                                         |        |
| Ctrl/U | Delete to beginning of line (saved to PASTE)                               |        |
| Ctrl/W | Refresh screen                                                             |        |
| Ctrl/Z | Return to line mode                                                        |        |

**GOLD+Ctrl/K** maps to DEL EOL. Ctrl/R refreshes (same as Ctrl/L / Ctrl/W), not a VMS "ruler". See §13.

## 7.5 Cursor / motion keys

| Key                                  | Action                                             | Source                         |
|--------------------------------------|----------------------------------------------------|--------------------------------|
| Up arrow                             | Move cursor up one line                            | <code>edt.cpp:3071</code>      |
| Down arrow                           | Move cursor down one line                          | <code>edt.cpp:3072</code>      |
| Left arrow                           | Move cursor left, wrapping to end of previous line | <code>edt.cpp:3073</code>      |
| Right arrow                          | Move cursor right, wrapping to start of next line  | <code>edt.cpp:3074</code>      |
| Home                                 | Beginning of line                                  | <code>edt.cpp:3075</code>      |
| End                                  | End of line                                        | <code>edt.cpp:3076</code>      |
| Backspace / DEL / Ctrl/H             | Delete character to the left                       | <code>edt.cpp:3077–3080</code> |
| <code>KEY_DC</code> (Delete forward) | DEL C — forward delete                             | <code>edt.cpp:3081–3082</code> |
| Return / Enter                       | Split line at cursor, advance to new row           | <code>edt.cpp:3083–3089</code> |
| Tab                                  | Insert spaces up to the next tab stop              | <code>edt.cpp:3212–3220</code> |

## 7.6 GOLD + letter / GOLD + arrow

| Chord      | Action                                             | Source                         |
|------------|----------------------------------------------------|--------------------------------|
| GOLD+Q     | QUIT — exit without saving                         | <code>edt.cpp:3130–3133</code> |
| GOLD+N     | Toggle line-number display                         | <code>edt.cpp:3159–3162</code> |
| GOLD+Up    | TOP                                                | <code>edt.cpp:3147–3149</code> |
| GOLD+Down  | BOTTOM                                             | <code>edt.cpp:3150–3152</code> |
| GOLD+Left  | Shift current line one space left (de-indent by 1) | <code>edt.cpp:3153–3155</code> |
| GOLD+Right | Shift current line one space right (indent by 1)   | <code>edt.cpp:3156–3158</code> |

## 7.7 LK201 editing keypad (E1–E6)

The DEC LK201 left editing keypad is mapped when ncurses delivers the corresponding `KEY_*` codes:

| Key                | ncurses                                      | Action                   |
|--------------------|----------------------------------------------|--------------------------|
| Prev Screen / PgUp | <code>KEY_PPAGE</code>                       | Page up                  |
| Next Screen / PgDn | <code>KEY_NPAGE</code>                       | Page down                |
| Insert Here        | <code>KEY_IC</code>                          | PASTE                    |
| Find               | <code>KEY_FIND</code>                        | FNDNXT                   |
| Select             | <code>KEY_SELECT</code>                      | SELECT                   |
| Command            | <code>KEY_COMMAND</code>                     | Line-mode command prompt |
| Home / End         | <code>KEY_HOME</code> / <code>KEY_END</code> | BOL / EOL                |
| Remove             | <code>KEY_DC</code>                          | DEL C                    |

Availability of `KEY_FIND` / `KEY_SELECT` / `KEY_COMMAND` / `KEY_IC` depends on the terminal and ncurses build; unmapped codes fall through.

## 7.8 Reference layer (not wired into Editor)

`edt.cpp` contains a parallel keypad emulator (`EDTKeypadEmulator` at `edt.cpp:4308–4509`) that decodes raw VT/xterm escape sequences into the explicit `EDTCommand` enum. It is **not** used by `Editor`'s main input loop, which dispatches directly inside `handle_screen_key()` / `handle_keypad_or_text()` / `handle_gold_key()`. The parallel layer is preserved as a reference for SSH terminals where ncurses fails to translate function keys, and as a target for tests that want to exercise the keypad in pure-C++ unit form.

## 8 Search and substitute

### 8.1 Search target syntax

`FIND text` accepts the needle in three forms:

| Form             | Example                         | Meaning                                         |
|------------------|---------------------------------|-------------------------------------------------|
| Bare token       | <code>FIND hello</code>         | Match the literal characters <code>hello</code> |
| Double-quoted    | <code>FIND "Hello World"</code> | Match including embedded spaces                 |
| Buffer reference | <code>FIND =name</code>         | Switch to the named buffer; <b>not</b> a search |

The classic VMS three-delimiter form `FIND /text/` is not supported.

### 8.2 Wildcard set

`TextBuffer::has_wildcards()` (`edt.cpp:862-870`) detects whether a needle uses any of the EDT wildcards. The matcher is `wmatch()` (`edt.cpp:882-928`) — a recursive backtracker.

| Token               | Meaning                                                                |
|---------------------|------------------------------------------------------------------------|
| <code>^</code>      | Start-of-line anchor (only meaningful at position 0 of the needle)     |
| <code>\$</code>     | End-of-line anchor (only meaningful at end of needle)                  |
| <code>*</code>      | Match any sequence of characters, including the empty sequence         |
| <code>?</code>      | Match exactly one character                                            |
| <code>[abc]</code>  | Match any one of <code>a</code> , <code>b</code> , or <code>c</code>   |
| <code>[^abc]</code> | Match anything except <code>a</code> , <code>b</code> , <code>c</code> |
| <code>[a-z]</code>  | Range form inside a character class                                    |
| <code>\\x</code>    | Backslash-escape the next character                                    |

If `has_wildcards()` returns false, the search falls back to the much faster `std::string::find` path.

### 8.3 Direction

The active search direction is the `direction_` member (`Direction::Advance` or `Direction::Backup`). It is changed by:

- `ADVANCE` / `BACKUP` line-mode commands (§5)
- KP4 / KP5 in screen mode
- `RESET` (resets to `ADVANCE`)

`FIND` always uses `ADVANCE` on first match. `FNDNXT` honours the active direction.

## 8.4 Case sensitivity

`search_exact_` (`edt.cpp:3890`) controls case folding. `SET SEARCH EXACT` makes it true (default); `SET SEARCH GENERAL` makes it false. The flag is forwarded to `find_forward`, `find_backward`, and `match_len_at`.

**Caveat.** The regression report (`EDT_Regression_Test_Report.pdf`) notes that the case-insensitive path was incomplete at the time of testing. Verify `find_forward()` and `find_backward()` at `edt.cpp:959-1045` before relying on `SET SEARCH GENERAL` to fold case.

## 8.5 Wrap-around

`FIND` and `FNDNXT` wrap. If the forward scan misses, the function restarts the scan from `(0, 0)`. If the backward scan misses, it restarts from the last column of the last line (`edt.cpp:3765-3775`).

## 8.6 Search flags (BEGIN / END / WORD)

`SET SEARCH BEGIN / END / WORD / NOWORD` (and companion `SET SEARCH ...` tokens) constrain `FIND` matches:

- **BEGIN** — match only at column 0 of a line
- **END** — match only when the match ends at end-of-line
- **WORD** — match only when surrounded by word delimiters (or ends of line); uses the same delimiter set as `WORD` motion

`SHOW SEARCH` reports the active flags.

## 8.7 Entity / word delimiters

In this build, whole-word `FIND` is `SET SEARCH WORD` (§8.6). `SET ENTITY WORD "delims" / SET WORD ...` configure the **delimiter set** used by `WORD` motion, `DEL W`, and `WORD`-constrained search.

## 8.8 SUBSTITUTE

`SUBSTITUTE [replacement]` does the following (`substitute_command()`):

1. Pull `replacement` from the argument, or — if absent — from the `PASTE` buffer.
2. Require `last_search_` to be set; otherwise emit `?No active search string`.
3. If `/GLOBAL` was set, hand off to `substitute_global()`.
4. Find the next match from the cursor (wrapping if needed).
5. With `/QUERY`, prompt `Substitute? [Y/N]:` on stdout; non-`Y` skips the match and advances.
6. Replace the matched text and advance the cursor to the next occurrence.

`SUBSTITUTE ALL` (or `/GLOBAL`) walks the entire buffer from `(0, 0)` and reports `<n> substitutions`.

## 8.9 Per-line repeat counts

VMS EDT's per-line substitution count (e.g. `SUBSTITUTE/old/new/3`, which substitutes up to three times per line) is not implemented.

## 9 EDTINI.EDT — startup file

### 9.1 Search order

`Editor::find_startup_file()` (`edt.cpp:2140-2165`) probes in this order and uses the first match:

1. `$EDT_INI` — the environment variable's value (treated as a file path).
2. `$HOME/EDT.INI` (uppercase form, preferred for VMS migrants).
3. `$HOME/edt.ini` (lowercase fallback).

On Windows, `%USERPROFILE%` is used if `$HOME` is unset (`edt.cpp:2148`).

The OpenVMS lookup chain (`SYS$LOGIN:EDTINI.EDT` then `SYS$LIBRARY:EDTINI.EDT`) does **not** apply on Linux. Use `EDT_INI` to point at any preferred file.

### 9.2 Format

The startup file is a plain text file of EDT line-mode commands, one per line. Anything `parse_command()` accepts is valid, including:

- `SET ... / SHOW ...` parameter tweaks
- `DEFINE KEY ...`
- `DEFINE MACRO ... AS "..."`
- Comments — lines beginning with `!`, `#`, or `;`
- Blank lines

The file is replayed by `run_command_file()` (`edt.cpp:2168-2185`) with `report=false` so output is suppressed. `EXIT/QUIT` commands encountered in EDT.INI are **deliberately ignored** to prevent a stray startup line from terminating the editor before the user gets a prompt; `CHANGE` is honoured and enters screen mode.

### 9.3 /INIT and /NOINIT

The `/INIT[=file]` and `/NOINIT` DCL qualifiers from VMS are mapped as follows in this build:

- To force a specific startup file, set `EDT_INI=path` in the environment before launching.
- To skip startup, set `EDT_INI` to a non-existent path (or an empty file) — the regression suite uses an empty `EDT_INI` to neutralise any user config (`EDT_Regression_Test_Report.pdf` §2).

## 9.4 Example EDT.INI

```
! EDT.INI - personal configuration for the * prompt
SET SEARCH GENERAL
SET NUMBERS
SET PROMPT "EDT> "
DEFINE KEY GOLD Q AS "QUIT"
DEFINE KEY GOLD E AS "EXIT"
DEFINE KEY CONTROL P AS "PASTE"
DEFINE KEY GOLD F AS "FIND"
DEFINE KEY GOLD S AS "SUBSTITUTE"
DEFINE MACRO HEADER AS "TOP; INSERT // Auto-generated"
```

## 10 Key definitions and macros

### 10.1 `DEFINE KEY` reference

```
DEFINE KEY [GOLD] [CONTROL] key AS "command"
```

- `GOLD` modifier — the key is a GOLD-shifted chord (PF1 followed by `key`).
- `CONTROL` or `CTRL` modifier — the key is held with Ctrl.
- `key` — one of:
  - `PF1`, `PF2`, `PF3`, `PF4` (also `F1–F4`)
  - `DELETE`
  - Any single printable ASCII character

The command string is replayed verbatim through `execute_command()` when the key is pressed in screen mode. Multiple verbs can be chained with `;` if the binding wants to do several things at once — although the cleanest pattern is `DEFINE KEY ... AS "MACRO-NAME"` plus a `DEFINE MACRO`.

### 10.2 Built-in key ids (canonical strings)

The id is the string produced by `normalize_key_spec()` (`edt.cpp:2841–2875`) and `key_id_from_input()` (`edt.cpp:2881–2895`). Both must produce the same id for a binding to match.

| Source key                      | Canonical id          |
|---------------------------------|-----------------------|
| <code>Q</code> , <code>q</code> | <code>Q</code>        |
| <code>GOLD + Q</code>           | <code>GOLD+Q</code>   |
| <code>CONTROL + P</code>        | <code>CTRL+P</code>   |
| <code>PF3</code>                | <code>PF3</code>      |
| <code>GOLD + PF3</code>         | <code>GOLD+PF3</code> |
| <code>DELETE</code> / Backspace | <code>DELETE</code>   |

### 10.3 Examples

```
DEFINE KEY GOLD Q AS "QUIT"
DEFINE KEY GOLD E AS "EXIT"
DEFINE KEY GOLD F AS "FIND"
DEFINE KEY PF3 AS "FNDNXT"
DEFINE KEY GOLD PF3 AS "FIND"
DEFINE KEY CONTROL P AS "PASTE"
DEFINE KEY GOLD I AS "INSERT Hello from key"
DEFINE KEY GOLD S AS "SUBSTITUTE done"
```

## 10.4 Function-name table

EDT internally identifies functions by **verb strings** rather than the classic VMS `FCT_XXX` symbols. The complete set of verb strings is the union of:

- Every entry in `parse_command()` (`edt.cpp:1468-1585`)
- Every `if (v == "...")` line in `dispatch_command()` (`edt.cpp:1739-1945`)
- Macro names registered via `DEFINE MACRO`

For convenience, the canonical verbs are:

|                    |               |                  |                |
|--------------------|---------------|------------------|----------------|
| ADVANCE            | APPEND        | BACKUP           | BOTTOM         |
| CHANGE             | CHGL          | CHGU             | CLEAR          |
| COPY               | CUT           | DEFINE-KEY       | DEFINE-MACRO   |
| DELETE             | DEL-C         | DEL-EOL          | DEL-L          |
| DEL-W              | EXIT          | FILL             | FIND           |
| FNDNXT             | GOTO-LINE     | HELP             | INCLUDE        |
| INSERT             | MOVE          | NEXT             | PASTE          |
| PREVIOUS           | PRINT         | QUIT             | REDO           |
| REPLACE            | RESET         | SELECT           | SET-ENTITY     |
| SET-INSERT         | SET-MODE-LINE | SET-NONUMBERS    | SET-NOQUIET    |
| SET-NOVERIFY       | SET-NOWRAP    | SET-NUMBERS      | SET-OVERSTRIKE |
| SET-PROMPT         | SET-QUIET     | SET-SEARCH-EXACT |                |
| SET-SEARCH-GENERAL | SET-TAB       | SET-VERIFY       | SET-WRAP       |
| SHOW-BUFFER        | SHOW-ENTITY   | SHOW-KEY         | SHOW-MACRO     |
| SHOW-NUMBERS       | SHOW-PROMPT   | SHOW-QUIET       | SHOW-SEARCH    |
| SHOW-TAB           | SHOW-VERIFY   | SHOW-WRAP        | SUBSTITUTE     |
| SUBSTITUTE-ALL     | TOP           | TYPE             | TYPECURRENT    |
| UND-C              | UND-L         | UND-W            | UNDO           |
| WRITE              |               |                  |                |

## 10.5 SHOW KEY

`SHOW KEY [pattern]` lists user-defined bindings whose id contains `pattern` (case-insensitive substring match). With no pattern, every user definition is dumped (`edt.cpp:1997-2010`).

## 10.6 DEFINE MACRO reference

```
DEFINE MACRO name AS "cmd; cmd; cmd"
```

or interactive form:

```
DEFINE MACRO name
Enter macro commands for NAME. End with END DEFINE or a blank line.
_TOP
_FIND TODO
_SUBSTITUTE DONE
_<blank line>
```

Macros are stored as `TextBuffer`s in the named-buffer map and a flag in `macro_names_` marks them as runnable. Typing the macro name at the prompt (or as the body of a `DEFINE KEY`) runs `try_execute_macro()` (`edt.cpp:2749-2775`), which:

- Refuses to recurse into a macro already on the running stack
- Iterates the macro's lines, skipping blanks and `!/#/;` comments
- Echoes each command if `SET VERIFY` is on
- Exits as soon as one of the commands returns `ExitSave` / `ExitNoSave` / `EnterScreenMode`

`SHOW MACRO` lists every defined macro. `SHOW MACRO name` shows just one.

## 11 Journal / recovery

A **text** command journal is implemented (not OpenVMS binary fidelity):

- Interactive line-mode commands append to `filename.jou`.
- `-recover` / `/RECOVER` replays that command file at startup when present.
- Screen-mode keystrokes are **not** fully journalled (CHANGE enter/leave markers are recorded).
- On a **clean EXIT** (successful save), the `.jou` file is **deleted**.
- No RMS-style fsync schedule.

The in-memory undo stack (`undo_stack_` / `redo_stack_`, capped at 100 entries by `max_undo_`) remains the primary interactive recovery tool. `UNDO` and `REDO` restore named buffers, cursor, selection, search state, and delete buffers.

## 12 Internals

### 12.1 Top-level structure of `edt.cpp`

The file is laid out top-to-bottom in this order (line numbers approximate, see `edt.cpp` for exact boundaries):

| Block                                | Lines     | Purpose                                                     |
|--------------------------------------|-----------|-------------------------------------------------------------|
| Doc-comment header                   | 1-130     | Overview, build instructions, implementation status         |
| Platform headers                     | 132-211   | ncurses vs PDCurses, POSIX vs Win32                         |
| Splash macros                        | 213-214   | Optional <code>SPLASH_SCREEN</code>                         |
| Keypad private codes                 | 233-248   | <code>EDT_KP0 ... EDT_KPE</code> above <code>KEY_MAX</code> |
| <code>enum class KeyCmd</code>       | 263-300   | Helper command names                                        |
| <code>class Keypad</code>            | 314-412   | Lightweight VT keypad mapper (not used by Editor)           |
| <code>class TextBuffer</code>        | 435-1161  | Line storage, SELECT mark, load/save, search                |
| <code>class Editor</code>            | 1165-3926 | Application core (line + screen, dispatch, undo)            |
| <code>parse_command_arg()</code>     | 3935-3960 | argv parser for <code>-command</code>                       |
| <code>redirect_tty()</code>          | 3967-4005 | Honour <code>\$VX_TTY</code> (POSIX; no-op on Windows)      |
| <code>show_splash_screen()</code>    | 4011-4126 | Optional splash ( <code>-DSPLASH_SCREEN</code> )            |
| <code>main()</code>                  | 4145-4190 | Entry point                                                 |
| <code>class EDTKeypadEmulator</code> | 4308-4509 | Reference keypad decoder (not wired in)                     |

### 12.2 Line storage

Every buffer is a `std::vector<std::string>`. Each element is one logical line **without** its terminating newline. Two invariants are enforced:

1. The vector is always non-empty (an empty buffer is `{ "" }`, `edt.cpp:1114`).
2. `modified_` is monotonically true until the next `save()` to the recorded `filename_`.

This data structure is dead simple — every insert/delete is an  $O(n)$  operation on the affected line, and line-level insert/delete is  $O(n)$  on the vector. There is no gap buffer, no piece table, no rope. For the file sizes EDT is asked to edit (source files, command scripts, log fragments) this is fast enough; the regression suite's 156 assertions complete in well under a second.

### 12.3 Terminal I/O

In line mode, all output goes through `std::cout`. There is no buffering trickery — `std::flush` is applied after each prompt (`edt.cpp:2095`).

In screen mode, ncurses owns the terminal. The frame is built by `draw_screen()` (`edt.cpp:3283-3361`):

- `erase()` clears the virtual screen.
- For each row in `[top_line_, top_line_ + text_rows)`, the buffer line is rendered, with optional line-number prefix and horizontal scroll offset.
- Characters inside the active SELECT span (`active_select_span()`) are highlighted with `A_REVERSE` (character-granular, not whole-line).
- A `---` ruler, a status line, and a message line occupy the bottom three rows.
- `move(y, x)` positions the hardware cursor; `refresh()` flushes the virtual screen to the terminal.

The application-keypad escape `\033=` is emitted on entry to screen mode (`edt.cpp:2990`); `\033>` on exit (`edt.cpp:3038`). These ensure the VT100 numeric keypad sends `ESC O p ... ESC O y` private codes rather than bare digits.

## 12.4 TTY raw-mode setup

`screen_mode()` calls, in order:

- `initscr()` — initialise ncurses, take control of the terminal.
- `raw()` — turn off line buffering and signal generation by the terminal driver.
- `noecho()` — don't echo keystrokes.
- `keypad(stdscr, TRUE)` — translate function/cursor keys to ncurses `KEY_*` constants.
- `nonl()` — don't translate `\n` to `\r\n` on output.
- `set_escdelay(25)` — shorten the wait for an ESC-sequence continuation byte from the default 1 s to 25 ms (POSIX/ncurses only).

## 12.5 Signal handling

`edt.cpp` does **not** install custom handlers for `SIGINT`, `SIGWINCH`, `SIGTERM`, etc. It relies on ncurses' default behaviour:

- `SIGWINCH` triggers ncurses' own resize handling; the next `getmaxyx()` call returns the new size and the frame redraws naturally on the next `draw_screen()`.
- `SIGINT` is consumed by ncurses' `raw()` mode (no terminal-level INT generation). `^C` is just a normal keystroke.

For batch jobs (line mode under stdin redirection), no signals are caught; the editor exits at EOF.

## 12.6 Memory limits

There is no explicit cap on:

- Number of lines per buffer
- Length of any single line
- Number of named buffers
- PASTE buffer size

The undo stack is the only bounded resource — 100 snapshots (`max_undo_`). For very large files the vector-of-strings layout means peak memory is roughly twice the file size, so multi-gigabyte edits will hit RAM limits before any internal cap.

## 13 Known divergences from VMS EDT

This list collates everything called out in the source comments (`edt.cpp` `KNOWN REMAINING GAPS`), the web page (`edt_web_text.txt`), and the regression report. Primary deferred item: **RMS / ;n versioning**.

### 13.1 Commands missing or stubbed

| Command                                   | Status          | Notes                                                                                                                                                                                                  |
|-------------------------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>RESEQUENCE</code>                   | Implemented     | Rewrites 8-column <code>NNNNN.00</code> prefixes; <code>/SEQUENCE=start:incr.</code>                                                                                                                   |
| <code>RETAIN / STORE TEXT</code>          | Not implemented | Use <code>COPY range TO =buffer</code> and <code>WRITE file BUFFER name.</code>                                                                                                                        |
| <code>XLATE</code>                        | Stub            | Reports unavailable on this host.                                                                                                                                                                      |
| <code>RANGE</code>                        | Not implemented | Range tokens are parsed but the <code>RANGE</code> verb itself is absent.                                                                                                                              |
| <code>FILES</code>                        | Not implemented | Use <code>NEXT / PREVIOUS</code> to walk the argv file list; <code>SHOW FILES</code> reports paths.                                                                                                    |
| <code>WRITE [range]</code>                | Implemented     | <code>WRITE file [range] [BUFFER name]</code> — see §5/WRITE.                                                                                                                                          |
| Signed line offsets ( <code>.</code> + 5) | Not implemented | <code>parse_line_ref()</code> accepts <code>.</code> , <code>CURRENT</code> , <code>BEGIN</code> , <code>FIRST</code> , <code>START</code> , <code>END</code> , <code>LAST</code> , and bare integers. |
| Nokeypad language                         | Implemented     | <code>SET KEYPAD NOKEYPAD + CHANGE</code> , or <code>CHANGE ;cmds</code> . Common alphabet; rare WPS-only entities may differ.                                                                         |

## 13.2 Qualifiers missing or partial

| Qualifier              | Status                         |
|------------------------|--------------------------------|
| /QUERY                 | SUBSTITUTE only                |
| /GLOBAL / /G           | SUBSTITUTE only                |
| /NOTYPE                | Call-wide                      |
| /BRIEF / /BRIEF:n      | TYPE / FIND (truncate display) |
| /SAVE                  | Post-dispatch save             |
| /STAY                  | FIND / SUBSTITUTE              |
| /DUPLICATE             | Soft-accepted on RESEQUENCE    |
| /SEQUENCE=start:incr   | RESEQUENCE numbering           |
| -readonly / /READ_ONLY | Blocks mutating commands       |
| -output= / /OUTPUT=    | Alternate EXIT save path       |
| -recover / /RECOVER    | Text .jou replay (§11)         |

## 13.3 SET/SHOW parameters

Most OpenVMS SET/SHOW names are accepted and reported. Soft scroll margins from `SET CURSOR top bottom` are applied in `adjust_view()`. Nokeypad is available via `SET KEYPAD NOKEYPAD` (§5/CHANGE, §13.1).

## 13.4 Keypad differences

- **GOLD+PF2** is BOTTOM (OpenVMS-aligned). PF2 alone still opens HELP.
- **GOLD+Enter** maps to SUBS ( `SUBSTITUTE` with PASTE as the replacement).
- **Ctrl/J** maps to LINEFEED (delete word left).
- **Ctrl/D** toggles ADVANCE / BACKUP.
- **Ctrl/K** is interactive DEFINE KEY; **GOLD+Ctrl/K** is DEL EOL.
- GOLD digit-repeat counts are supported when `SET REPEAT` is on.
- SPECINS ( `GOLD+3` ) accepts a decimal ASCII code when the prompt is digits.
- LK201 editing keys: PgUp/PgDn, Insert→PASTE, Find→FNDNXT, Select, Command→prompt (when nurses delivers the codes).
- **Ctrl/R** refreshes the screen (same as Ctrl/L / Ctrl/W), not a ruler.

## 13.5 Search and substitute

- `SET SEARCH GENERAL / EXACT` and BEGIN/END/WILD/WORD flags drive FIND.
- `SUBSTITUTE/old/new/` three-delimiter form is implemented.

- `ADVANCE` / `BACKUP` / `RESET` write to `message_` (screen) and also print direction changes in line mode for ADVANCE/BACKUP.
- Per-line substitution counts ( `/old/new/3` ) are not implemented (§8.9).

## 13.6 File system / RMS (deferred)

- Host text files only. **RMS record formats are intentionally deferred.**
- No VMS file versioning (no `;n` suffix machinery) — deferred with RMS.
- `/READ_ONLY` is enforced for mutating commands.
- `INCLUDE` and `WRITE` accept ordinary POSIX paths.

## 13.7 Journal / recovery

See §11. Text `file.jou` append + `-recover` replay; `.jou` deleted on clean EXIT. Not a byte-identical OpenVMS journal. UNDO/REDO is in-memory, capped at 100 levels.

## 13.8 DCL / command procedures

No DCL integration beyond the argv parser. `EDIT/EDT` is a wrapper issued by VX/RT's DCL emulator; from this binary's perspective the qualifiers are passed as ordinary POSIX-style flags ( `-command`, `-output=`, `-readonly`, `-recover` ).

## 13.9 Documented in `edt.cpp` itself

The header "KNOWN REMAINING GAPS" summary:

- RMS record formats / OpenVMS `;n` file versioning are not modeled (explicitly deferred).
- Journal/RECOVER is a text command journal ( `file.jou` ), not a byte-identical OVMS binary journal; screen-mode keystrokes are not fully reconstructed. Clean EXIT deletes the `.jou` when present.
- Nokeypad covers the common alphabet; rare WPS-only entities may differ. SET KEYPAD WPS aliases to EDT with a notice.
- EDT wildcard patterns beyond `%` / `*` remain limited.



## 14 Files this build cares about

| Path                                                                                      | Purpose                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/Volumes/s7/s7tools/vxtools85/edt/edt.cpp</code>                                    | Single-translation-unit source for the editor.                                                                                                                          |
| <code>/Volumes/s7/s7tools/vxtools85/edt/makefile</code>                                   | Build rules. Calls<br><code>g++ -std=c++17 -Wall -Wextra -pedantic -g edt.cpp -lncurses -o \$(BINDIR)/edt.exe</code> .                                                  |
| <code>\$(BINDIR)/edt.exe</code>                                                           | Linked binary (location set by <code>../makevars</code> ). On most build hosts this resolves to <code>/Volumes/s7/s7tools/vxtools85/build/&lt;arch&gt;/edt.exe</code> . |
| <code>\$HOME/EDT.INI</code> (or <code>\$EDT_INI</code> , or <code>\$HOME/edt.ini</code> ) | Per-user startup file (§9).                                                                                                                                             |
| <code>file.jou</code>                                                                     | Text command journal (§11); deleted on clean EXIT.                                                                                                                      |
| <code>/Volumes/s7/s7tools/vxtools85/edt/edt_test_suite/</code>                            | Python regression harness ( <code>run_edt_tests.py</code> ).                                                                                                            |
| <code>/Volumes/s7/s7tools/vxtools85/edt/edt_regression.sh</code>                          | Bash regression suite (156 assertions).                                                                                                                                 |
| <code>/Volumes/s7/s7tools/vxtools85/edt/test_input.txt</code>                             | Canonical 10-line input used by the bash regression suite.                                                                                                              |
| <code>/Volumes/s7/s7tools/vxtools85/edt/test_edt.txt</code>                               | Manual test checklist for screen mode.                                                                                                                                  |
| <code>/Volumes/s7/s7tools/vxtools85/edt/docs/EDT_User_Manual.txt</code>                   | Sector7 prose user manual.                                                                                                                                              |
| <code>/Volumes/s7/s7tools/vxtools85/edt/EDT_Introduction.docx</code>                      | Sector7 intro document.                                                                                                                                                 |
| <code>/Volumes/s7/s7tools/vxtools85/edt/EDT_Keypad_Reference.docx</code>                  | Sector7 keypad cheat sheet.                                                                                                                                             |
| <code>/Volumes/s7/s7tools/vxtools85/edt/OVMS_EDT_REF.pdf</code>                           | DEC OpenVMS EDT reference manual (authoritative for VMS-spec).                                                                                                          |
| <code>/Volumes/s7/s7tools/vxtools85/edt/docs/EDT_Regression_Test_Report.md</code>         | Line-mode regression summary (28 passed, 2026-07-15).                                                                                                                   |
| <code>/Volumes/s7/s7tools/vxtools85/edt/docs/edt_web_text.txt</code>                      | Web-page-style summary used for the public Sector7 page.                                                                                                                |
| <code>/Volumes/s7/s7tools/vxtools85/edt/docs/EDT_REFERENCE.md</code>                      | This document.                                                                                                                                                          |
|                                                                                           | PDF render of this document.                                                                                                                                            |

| Path                                                     | Purpose                       |
|----------------------------------------------------------|-------------------------------|
| /Volumes/s7/s7tools/vxtools85/edt/docs/EDT_REFERENCE.pdf |                               |
| /Volumes/s7/s7tools/vxtools85/edt/docs/EDT_CHANGELOG.md  | Release change log (V1.02-0). |
| /Volumes/s7/s7tools/vxtools85/edt/docs/EDT_CHANGELOG.pdf | PDF render of the change log. |

## 15 End notes

### 15.1 Build

POSIX:

```
g++ -std=c++17 -Wall -Wextra -pedantic -O2 edt.cpp -lncurses -o edt
```

Or via the included makefile:

```
make
```

Windows 11 (MSYS2 / MinGW-w64, PDCurses):

```
g++ -std=c++17 -Wall -Wextra -O2 edt.cpp -lpdcurses -o edt.exe
```

Windows 11 (MSVC + vcpkg pdcurses):

```
cl /std:c++17 /O2 edt.cpp pdcurses.lib /Fe:edt.exe
```

### 15.2 Regression suite

The bash regression script is the primary CI gate:

```
./edt_regression.sh
```

It produces a per-section PASS/SKIP report and exits 0 iff all assertions pass.

For ad-hoc Python-driven testing:

```
python3 edt_test_suite/run_edt_tests.py --source ./edt.cpp  
python3 edt_test_suite/run_edt_tests.py --source ./edt.cpp --include-extended
```

## 15.3 Source statistics

|                        |                                                                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Source file            | <code>/Volumes/s7/s7tools/vxtools85/edt/edt.cpp</code>                                                                                       |
| Size                   | 198,269 bytes (~193 KB)                                                                                                                      |
| Lines                  | 4511                                                                                                                                         |
| Classes                | <code>TextBuffer</code> , <code>Editor</code> , <code>Keypad</code> , <code>EDTKeypadEmulator</code>                                         |
| Enums                  | <code>KeyCmd</code> , <code>EDTKey</code> , <code>EDTCommand</code> , <code>Editor::Direction</code> ,<br><code>Editor::CommandResult</code> |
| Public entry point     | <code>main()</code> at <code>edt.cpp:4145</code>                                                                                             |
| Headers / dependencies | C++17 standard library plus ncurses (POSIX) / PDCurses (Windows).                                                                            |
| Copyright              | <code>Sector7 USA LLC 2026 (edt.cpp:4)</code>                                                                                                |

## 15.4 Acknowledgements

EDT was originally written by Digital Equipment Corporation for the VAX/VMS operating system, beginning in the early 1980s. The GOLD-key/keypad model documented here is unchanged from the original VT100 / VT220 / LK201 hardware. The Sector7 reimplementations preserves the user-visible model while replacing the back end with a modern C++17 / ncurses runtime suitable for Linux, macOS, and Windows 11. The original DEC reference ([OVMS\\_EDT\\_REF.pdf](#)) remains the authoritative source for VMS-specific behaviour; this document is the engineering reference for the Sector7 build.

## Appendix A — Status line and message reference

These strings appear in the screen-mode status / message lines (`edt.cpp:3345-3354` and the various `message_` assignments).

| Message                                      | Triggered by                                                                              |
|----------------------------------------------|-------------------------------------------------------------------------------------------|
| EDIT[<name>]                                 | Status line, where <name> is the active buffer.                                           |
| ADVANCE / BACKUP                             | Status line, current search direction.                                                    |
| OVR                                          | Status line, when <code>overstrike_</code> is true.                                       |
| SELECT                                       | Status line, when a SELECT range is active; also message after <code>SELECT</code> chord. |
| * col+N                                      | Modified flag and horizontal scroll indicator.                                            |
| GOLD                                         | After PF1 is pressed; cleared when next key is consumed.                                  |
| TOP / BOTTOM                                 | After motion to first/last line.                                                          |
| RESET                                        | After RESET / GOLD+ . .                                                                   |
| Line numbers ON / Line numbers OFF           | After GOLD+N / SET NUMBERS / SET NONUMBERS .                                              |
| Insert mode / Overstrike                     | After Ctrl/A / SET INSERT / SET OVERSTRIKE .                                              |
| Buffer <NAME>                                | After <code>FIND =name</code> .                                                           |
| Range deleted                                | After multi-line delete.                                                                  |
| Selected range cut / Selected range appended | After CUT/APPEND consumed a SELECT range.                                                 |
| Paste buffer empty                           | After REPLACE attempted with empty PASTE.                                                 |
| Replaced from PASTE buffer                   | After REPLACE succeeded.                                                                  |
| No deleted line                              | UND L with empty <code>delete_line_buffer_</code> .                                       |
| Filled                                       | After <code>FILL</code> .                                                                 |
| Found: <needle>                              | After successful FIND / FNDNXT.                                                           |
| String not found: <needle>                   | After FIND / FNDNXT miss.                                                                 |
| Substituted                                  | After SUBSTITUTE.                                                                         |
| <n> substitutions                            | After SUBSTITUTE ALL / /GLOBAL .                                                          |
| UNDO / REDO                                  | After successful undo/redo.                                                               |
| Nothing to undo / Nothing to redo            | When stacks are empty.                                                                    |
| Command mode                                 | After Ctrl/Z, just before screen mode tears down.                                         |
| Undefined GOLD key                           | When a GOLD chord is unbound and unrecognised.                                            |

## Appendix B — Line-mode prompt error reference

Strings emitted on stdout by `dispatch_command()` and friends; each is prefixed with `?` per the EDT convention. Read the source line cited to see the exact emission point.

| Message                             | Source                         |
|-------------------------------------|--------------------------------|
| ?Invalid line number: <token>       | edt.cpp:1755                   |
| ?Invalid range: <token>             | edt.cpp:1655, 1667, 2180, 2443 |
| ?Unrecognized command: <text>       | edt.cpp:1932                   |
| ?No active search string            | edt.cpp:2399, 2445             |
| ?String not found: <needle>         | edt.cpp:2408                   |
| ?Missing buffer name                | edt.cpp:2225                   |
| ?Missing destination buffer         | edt.cpp:2325                   |
| ?Syntax: COPY range TO =buffer      | edt.cpp:2312                   |
| ?Unable to open include file <path> | edt.cpp:2342                   |
| ?Unable to write <path>             | edt.cpp:2380, 3376             |
| ?WRITE requires a file name         | edt.cpp:2368                   |
| ?Invalid macro name                 | edt.cpp:2708                   |
| ?Macro has no commands              | edt.cpp:2730                   |
| ?DEFINE MACRO syntax: ...           | edt.cpp:2698                   |
| ?DEFINE KEY syntax: ...             | edt.cpp:2990                   |
| ?Invalid key specification          | edt.cpp:2823                   |
| ?Recursive macro invocation: <name> | edt.cpp:2754                   |
| ?Macro buffer missing: <name>       | edt.cpp:2759                   |
| ?No such macro: <name>              | edt.cpp:2809                   |
| ?Cannot create temp file            | edt.cpp:2418, 2425             |
| ?Print failed                       | edt.cpp:2635                   |
| ?Unable to save <path>              | edt.cpp:1941                   |
| ?No more files                      | edt.cpp:2017                   |
| ?Already at first file              | edt.cpp:2037                   |
| ?Nothing to undo / ?Nothing to redo | edt.cpp:1665, 1686             |
| ?Key not defined: <name>            | edt.cpp:2005                   |

## Appendix C — Quick reference card

### Line mode

|             |                                                                                                                                                                                                                                                                                                                                                                 |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Motion      | TOP BOTTOM <n> . CURRENT BEGIN END                                                                                                                                                                                                                                                                                                                              |
| Display     | TYPE [range] T [range] TYPE WHOLE /BRIEF[:n]                                                                                                                                                                                                                                                                                                                    |
| Edit        | INSERT text INSERT (multiline) DELETE [range] FILL<br>TAB TAB ADJUST [range] RESEQUENCE [/SEQUENCE=n:i]                                                                                                                                                                                                                                                         |
| Search      | FIND text FIND =buffer FNDNXT<br>SET SEARCH EXACT GENERAL WORD BEGIN END<br>ADVANCE BACKUP RESET                                                                                                                                                                                                                                                                |
| Substitute  | SUBSTITUTE [text] SUBSTITUTE ALL<br>SUBSTITUTE/QUERY SUBSTITUTE/GLOBAL                                                                                                                                                                                                                                                                                          |
| Clipboard   | SELECT CUT APPEND PASTE REPLACE                                                                                                                                                                                                                                                                                                                                 |
| Undo        | UNDO U REDO                                                                                                                                                                                                                                                                                                                                                     |
| Buffers     | SHOW BUFFER FIND =name CLEAR [name]<br>COPY range TO =name MOVE range TO =name                                                                                                                                                                                                                                                                                  |
| File        | INCLUDE file WRITE file [range] [BUFFER name]<br>NEXT PREVIOUS EXIT QUIT                                                                                                                                                                                                                                                                                        |
| Version     | SHOW VERSION VERSION                                                                                                                                                                                                                                                                                                                                            |
| Settings    | SET NUMBERS / NONUMBERS SHOW NUMBERS<br>SET SEARCH EXACT / GENERAL / WORD SHOW SEARCH<br>SET OVERSTRIKE / INSERT<br>SET QUIET / NOQUIET / VERIFY / NOVERIFY<br>SET WRAP / NOWRAP<br>SET TAB n SHOW TAB<br>SET CURSOR top bottom SHOW CURSOR<br>SET KEYPAD EDT NOKEYPAD WPS SHOW KEYPAD<br>SET PROMPT "... " SHOW PROMPT<br>SET ENTITY WORD "delims" SHOW ENTITY |
| Keys/macros | DEFINE KEY [GOLD] [CONTROL] key AS "cmd"<br>DEFINE MACRO name AS "cmd; cmd"<br>SHOW KEY [pattern] SHOW MACRO [name]                                                                                                                                                                                                                                             |
| Mode        | CHANGE C CHANGE ;nokeypad-cmds<br>SET MODE CHANGE SET MODE LINE                                                                                                                                                                                                                                                                                                 |

## Screen mode

```
PF1=GOLD  PF2=HELP  PF3=FNDNXT  PF4=DEL L
GOLD+PF2=BOTTOM  GOLD+PF3=FIND  GOLD+PF4=UND L

KP0 LINE      GOLD+KP0 OPEN LINE
KP1 WORD      GOLD+KP1 CHNGCASE
KP2 EOL       GOLD+KP2 DEL EOL
KP3 CHAR      GOLD+KP3 SPECINS
KP4 ADVANCE   GOLD+KP4 BOTTOM
KP5 BACKUP    GOLD+KP5 TOP
KP6 CUT       GOLD+KP6 PASTE
KP7 PAGE      GOLD+KP7 COMMAND
KP8 SECT      GOLD+KP8 FILL
KP9 APPEND    GOLD+KP9 REPLACE
KP- DEL W     GOLD+KP- UND W
KP, DEL C     GOLD+KP, UND C
KP. SELECT    GOLD+KP. RESET

Ctrl/A toggle insert/overstrike  Ctrl/Z return to line mode
Ctrl/B beginning of line         Ctrl/E end of line
Ctrl/D toggle ADVANCE/BACKUP     Ctrl/F FNDNXT
Ctrl/K DEFINE KEY                GOLD+Ctrl/K DEL EOL
Ctrl/J LINEFEED (del word left)  Ctrl/C abort GOLD/select
Ctrl/L Ctrl/R Ctrl/W refresh     Ctrl/T toggle case
Ctrl/U delete to BOL (to PASTE)

GOLD+Q  QUIT          GOLD+N  toggle line numbers
GOLD+Up TOP          GOLD+Dn  BOTTOM
GOLD+Lf shift left   GOLD+Rt  shift right

LK201: PgUp/PgDn  Insert=PASTE  Find=FNDNXT  Select  Command=prompt
```

## Qualifiers

```
/QUERY  /GLOBAL  /G  /NOTYPE  /BRIEF[:n]  /SAVE  /STAY
/SEQUENCE=start:incr
```

## Startup / CLI

```
$EDT_INI          (explicit override)
$HOME/EDT.INI     (preferred path)
$HOME/edt.ini     (fallback)

edt file [-command=...] [-output=...] [-readonly] [-recover]
```

— END OF REFERENCE —